

Source Code Security Review

Application Vulnerability Assessment, Threat Modeling & Defensive Remediation Guide

Target Application:	Billing & Invoicing System	Assessment Date:	September 2026
Repository Scope:	e:/ocore/billing-system	Standards Applied:	OWASP Top 10 (2021), CWE / SANS 25
Tech Stack:	Python (Flask, SQLAlchemy, ReportLab) + React 18	Overall Posture:	CRITICAL / HIGH RISK

1. Executive Summary

A rigorous static source code security audit of the **Billing & Invoicing System** was conducted. The review evaluated backend architecture, database models, session management, PDF generation pipelines, executable packaging configurations, and frontend API interactions. The assessment identified **12 distinct vulnerabilities**, including **2 Critical** and **3 High** severity issues requiring immediate remediation.

2 CRITICAL	3 HIGH	4 MEDIUM	3 LOW	12 TOTAL FINDINGS
---------------	-----------	-------------	----------	----------------------

2. Vulnerability Findings Matrix

ID	Finding Title	Severity	CWE Classification	Affected Location
SEC-01	Broken Access Control on All Billing Endpoints	Critical	CWE-306, CWE-639	backend/app.py:128-253
SEC-02	Hardcoded Cryptographic SECRET_KEY	Critical	CWE-798, CWE-321	backend/app.py:44
SEC-03	Hardcoded Default Admin Credentials & UI Exposure	High	CWE-1188, CWE-798	backend/app.py & Login.jsx
SEC-04	Unsanitized Input in PDF Generator (DoS & XML Injection)	High	CWE-79, CWE-91	backend/pdf_generator.py:107-154
SEC-05	Production Execution Path with Enabled Flask Debug Mode	High	CWE-215, CWE-489	backend/app.py:279
SEC-06	Missing CSRF Protection & Permissive CORS Credentials	Medium	CWE-352, CWE-942	backend/app.py:48
SEC-07	Absence of Rate Limiting on Authentication Routes	Medium	CWE-307	backend/app.py:65-109
SEC-08	Inadequate Password Complexity & Length Policy	Medium	CWE-521	backend/app.py:94-95
SEC-09	Missing Financial & Payload Business Logic Validation	Medium	CWE-20, CWE-840	backend/app.py:149-186
SEC-10	Missing HTTP Security Headers & Cookie Flags	Medium	CWE-693, CWE-614	backend/app.py
SEC-11	Unencrypted Local Database & PDF Storage at Rest	Low	CWE-311	backend/data/

SEC-12	Raw Internal Exception Details Disclosed to Client	Low	CWE-209	backend/app.py:245
--------	--	-----	---------	--------------------

3. Detailed Vulnerability Analyses & Remediations

SEC-01: Broken Access Control on All Billing Endpoints

Severity: Critical (CVSS: 9.8) | **Classification:** CWE-306 (Missing Authentication), CWE-639 (Insecure Direct Object Reference)

Location: backend/app.py (Lines 128-253) & backend/models.py (Lines 29-48)

Description & Mechanics: The backend contains authentication logic (/api/login, /api/register), but none of the actual invoice management routes verify session state. Any unauthenticated caller can query all customer records, generate invoices, and download PDF documents. Furthermore, the Bill model lacks a user_id foreign key, meaning all authenticated users can access and modify every other user's billing records.

Recommended Defensive Implementation:

```
# 1. Create session verification decorator:
from functools import wraps
def login_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if 'user_id' not in session:
            return jsonify({'error': 'Authentication required'}), 401
        return f(*args, **kwargs)
    return decorated

# 2. Protect routes & isolate per user:
@app.route('/api/bills', methods=['GET'])
@login_required
def get_bills():
    user_id = session['user_id']
    bills = Bill.query.filter_by(user_id=user_id).order_by(Bill.created_at.desc()).all()
    return jsonify([b.to_dict() for b in bills]), 200
```

SEC-02: Hardcoded Cryptographic SECRET_KEY

Severity: Critical (CVSS: 9.1) | **Classification:** CWE-798 (Hard-coded Credentials), CWE-321 (Hard-coded Cryptographic Key)

Location: backend/app.py (Line 44)

Description & Mechanics: app.config['SECRET_KEY'] is hardcoded to 'billing-system-secret-key-2026' in source code. Flask signs client session cookies using this key. Any attacker with repository access or reading this static value can forge valid administrative session cookies and achieve complete authentication bypass.

Recommended Defensive Implementation:

```
import secrets

def get_or_create_secret_key(data_dir):
    env_key = os.environ.get('SECRET_KEY')
    if env_key:
        return env_key
    key_file = os.path.join(data_dir, '.secret_key')
    if os.path.exists(key_file):
        with open(key_file, 'r', encoding='utf-8') as f:
            return f.read().strip()
    new_key = secrets.token_hex(32)
    with open(key_file, 'w', encoding='utf-8') as f:
        f.write(new_key)
    return new_key

app.config['SECRET_KEY'] = get_or_create_secret_key(data_dir)
```

SEC-03: Hardcoded Default Admin Credentials & UI Exposure

Severity: High (CVSS: 8.2) | **Classification:** CWE-1188 (Insecure Default Initialization), CWE-798 (Hard-coded Credentials)

Location: backend/app.py (Lines 56-61) & frontend/src/components/Login.jsx

Description & Mechanics: The database initialization routine automatically provisions an admin account with password 'admin123'. The frontend pre-populates these credentials into form state and displays 'Default Demo Account: admin / admin123' on the login interface, allowing trivial takeover of all newly deployed instances.

Recommended Defensive Implementation:

```
# 1. In backend/app.py: Remove automatic credential seeding from init_db():
def init_db():
    with app.app_context():
        db.create_all()
        # Enforce account creation through registration or CLI setup only

# 2. In Login.jsx: Clear default initial state & delete hardcoded demo credentials banner:
const [username, setUsername] = useState('');
const [password, setPassword] = useState('');
```

SEC-04: Unsanitized Input in ReportLab PDF Generation (DoS & XML Injection)

Severity: High (CVSS: 7.5) | **Classification:** CWE-79 (Improper Neutralization of Input), CWE-91 (XML Injection)

Location: backend/pdf_generator.py (Lines 107-154)

Description & Mechanics: ReportLab's Paragraph class parses content using an internal XML parser. User strings from customer details and item names are formatted directly into Paragraph objects without escaping. Characters such as '&' (e.g. 'Johnson & Sons') or unclosed brackets trigger xml.parsers.expat.ExpatError, crashing the request with HTTP 500 and causing Denial of Service. Attackers can also inject arbitrary formatting or hyperlinks.

Recommended Defensive Implementation:

```
from xml.sax.saxutils import escape as xml_escape

def safe_text(val):
    return xml_escape(str(val)) if val else ''

# Sanitize all dynamic fields before creating Paragraphs:
clean_name = safe_text(item.get('name', 'Item'))
table_data.append([Paragraph(clean_name, normal_style), ...])

safe_customer = safe_text(bill_data.get('customer_name', ''))
story.append(Paragraph(f'<b>{safe_customer}</b>', normal_style))
```

SEC-05: Production Execution Path with Enabled Flask Debug Mode

Severity: High (CVSS: 8.6) | **Classification:** CWE-215 (Sensitive Info in Debugging Code), CWE-489 (Active Debug Code)

Location: backend/app.py (Line 279)

Description & Mechanics: The server execution block sets debug=True hardcoded. When launched via python app.py, the interactive Werkzeug debugger is activated. The debugger exposes a web-based Python execution console upon any server exception, which can lead to arbitrary remote code execution on the underlying host.

Recommended Defensive Implementation:

```
if __name__ == '__main__':
    init_db()
    port = int(os.environ.get('PORT', 5000))
    # Default debug mode to False; activate only if explicitly set in environment
    debug_mode = os.environ.get('FLASK_DEBUG', 'False').lower() in ('true', '1', 't')
    app.run(host='127.0.0.1', port=port, debug=debug_mode)
```

SEC-06: Missing CSRF Protection & Permissive CORS Credentials

Severity: Medium (CVSS: 6.5) | **Classification:** CWE-352 (Cross-Site Request Forgery), CWE-942 (Permissive CORS)

Location: backend/app.py (Line 48)

Description & Mechanics: The application enables CORS with `supports_credentials=True` and uses cookies for authentication, but implements no anti-CSRF token verification. A malicious website visited by an authenticated user could issue cross-site POST requests to manipulate billing records or invalidate sessions.

Recommended Defensive Implementation:

```
# Configure strict session cookie flags:
app.config['SESSION_COOKIE_SAMESITE'] = 'Lax'
app.config['SESSION_COOKIE_HTTPONLY'] = True
app.config['SESSION_COOKIE_SECURE'] = os.environ.get('ENV') == 'production'

# For production web deployments, enforce Flask-WTF CSRFProtect
```

SEC-07: Absence of Rate Limiting on Authentication Routes

Severity: Medium (CVSS: 5.3) | **Classification:** CWE-307 (Improper Restriction of Excessive Authentication Attempts)

Location: backend/app.py (Lines 65-109)

Description & Mechanics: The login and registration endpoints do not throttle incoming requests. Attackers can execute rapid automated brute-force attacks against user passwords or flood the database with bogus user registrations.

Recommended Defensive Implementation:

```
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

limiter = Limiter(get_remote_address, app=app, default_limits=['300/hour'])

@app.route('/api/login', methods=['POST'])
@limiter.limit('5 per minute')
def login():
    # ...
```

SEC-08: Inadequate Password Complexity & Length Policy

Severity: Medium (CVSS: 5.3) | **Classification:** CWE-521 (Weak Password Requirements)

Location: backend/app.py (Lines 94-95)

Description & Mechanics: The registration handler enforces only a 4-character minimum password length (`len(password) < 4`), permitting short, highly vulnerable passwords that can be cracked via brute-force or dictionary lookup within seconds.

Recommended Defensive Implementation:

```
if len(password) < 8:
    return jsonify({'error': 'Password must be at least 8 characters long'}), 400
if not any(c.isdigit() for c in password) or not any(c.isalpha() for c in password):
    return jsonify({'error': 'Password must contain both letters and digits'}), 400
```

SEC-09: Missing Financial & Payload Business Logic Validation

Severity: Medium (CVSS: 5.3) | **Classification:** CWE-20 (Improper Input Validation), CWE-840 (Business Logic Errors)

Location: backend/app.py (Lines 149-186)

Description & Mechanics: The invoice creation route does not validate numeric boundaries: unit_price and qty can be negative, allowing negative invoice totals. The tax_rate can be set to unbounded values. Additionally, there is no upper limit on the items array size, leaving the system vulnerable to memory exhaustion DoS when generating large PDF documents.

Recommended Defensive Implementation:

```
app.config['MAX_CONTENT_LENGTH'] = 2 * 1024 * 1024 # 2MB payload cap

if len(items) > 100:
    return jsonify({'error': 'Invoice items list cannot exceed 100 lines'}), 400
if qty <= 0 or unit_price < 0:
    return jsonify({'error': 'Quantity must be > 0 and unit price >= 0'}), 400
if not (0.0 <= tax_rate <= 100.0):
    return jsonify({'error': 'Tax rate must be between 0 and 100%'}), 400
```

SEC-10: Missing HTTP Security Headers & Cookie Flags

Severity: Medium (CVSS: 4.8) | **Classification:** CWE-693 (Protection Mechanism Failure), CWE-614 (Missing Cookie Secure Flag)

Location: backend/app.py

Description & Mechanics: The application does not emit standard HTTP defensive headers such as Content-Security-Policy (CSP), X-Frame-Options, or X-Content-Type-Options, leaving clients vulnerable to clickjacking and MIME-sniffing.

Recommended Defensive Implementation:

```
@app.after_request
def apply_security_headers(response):
    response.headers['X-Content-Type-Options'] = 'nosniff'
    response.headers['X-Frame-Options'] = 'DENY'
    response.headers['Referrer-Policy'] = 'strict-origin-when-cross-origin'
    response.headers['Content-Security-Policy'] = (
        "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline';"
    )
    return response
```

SEC-11: Unencrypted Local Database & PDF Storage at Rest

Severity: Low (CVSS: 3.3) | **Classification:** CWE-311 (Missing Encryption of Sensitive Data)

Location: backend/data/billing.db & backend/data/invoices/

Description & Mechanics: Invoices and customer PII (names, phone numbers, addresses, financial figures) are stored in plaintext SQLite and unencrypted PDF files. Any user with read access to the local workstation filesystem can extract sensitive data.

Recommended Defensive Implementation:

```
# Remediation:
# 1. Ensure Windows DACLS restrict data/ folder to the owner's account.
# 2. For enterprise deployments, integrate SQLCipher (pysqlcipher3) for transparent AES-256 DB encryption.
```

SEC-12: Raw Internal Exception Details Disclosed to Client

Severity: Low (CVSS: 3.1) | **Classification:** CWE-209 (Generation of Error Message Containing Sensitive Information)

Location: backend/app.py (Line 245)

Description & Mechanics: In download_bill_pdf, the exception block returns jsonify({'error': f'Failed to generate PDF: {str(e)}'}), 500. Exposing raw exception strings leaks internal library versions, file system structures, or parser internals.

Recommended Defensive Implementation:

```
except Exception as e:
    app.logger.error('PDF invoice generation failure: %s', str(e), exc_info=True)
    return jsonify({'error': 'Failed to generate PDF invoice. Please check the system logs.'}), 500
```

4. Strategic Remediation Roadmap

Phase & Priority	Target Actions	Timeline
Phase 1: Immediate Critical Fixes	- Enforce @login_required session guard on all /api/bills* routes. - Associate Bill records with user_id for multi-tenant isolation. - Replace hardcoded SECRET_KEY with securely generated persistent key.	24 - 48 Hours
Phase 2: High Priority Hardening	- XML-escape all user input passed to ReportLab in pdf_generator.py. - Remove default admin seed credentials and UI pre-population. - Default debug=False in app.py. - Validate financial boundaries (qty > 0, unit_price >= 0, tax_rate <= 100).	Current Sprint
Phase 3: Defense in Depth Operational Security	- Emit defensive HTTP security headers (CSP, X-Frame-Options, nosniff). - Integrate Flask-Limiter rate limiting on authentication routes. - Upgrade password policy to minimum 8 characters with complexity checks. - Apply filesystem DACL permission restrictions on data/ folder.	Next Release

5. Audit Conclusion & Sign-Off

The Billing & Invoicing System demonstrates a clean modular structure and successfully integrates SQLAlchemy ORM, ReportLab document generation, and Waitress WSGI hosting. However, the identified critical deficiencies—specifically unauthenticated billing access, hardcoded cryptographic keys, and unescaped XML parsing in the PDF engine—pose immediate security risks in any networked environment.

By applying the remediation code snippets provided in this report, engineering teams can resolve all critical and high-severity vectors, ensuring robust compliance with industry security baselines.

Report Generated By: Antigravity Security Audit Agent	Date: 2026-09-27
Target Directory: e:/ocore/billing-system	Status: Remediation Recommended Prior to Production